ESE 546

HOMEWORK 2

JINHAO YOU [YOU7@SEAS.UPENN.EDU]

Solution 1 (Time spent: 3 hours)**.** Your solution goes here.

Problem a. When BN is placed before ReLU (i.e., Conv $\rightarrow$ BN $\rightarrow$ ReLU), which is the standard and widely adopted configuration, BN first normalizes the convolutional output to have zero mean and unit variance, followed by the nonlinear transformation. This arrangement ensures that the inputs to the ReLU are well-conditioned, roughly centered around zero, allowing both positive and negative activations to be preserved. As a result, the network maintains a balanced activation distribution, avoids dead neurons, and exhibits more stable gradients, facilitating faster and more reliable convergence during training.

Conversely, placing ReLU before BN (Conv $\rightarrow$ ReLU $\rightarrow$ BN) often degrades performance. Since ReLU truncates all negative activations to zero, the subsequent BN layer receives inputs with a non-symmetric, non-zero-centered distribution. BN then attempts to re-normalize this skewed input, often shifting the zero activations toward negative values, which in turn diminishes the sparsity benefits of ReLU and can lead to reduced representational capacity. Empirically, this configuration tends to slow convergence and decrease overall accuracy. Therefore, the BN-before-ReLU ordering is generally preferred for deep architectures.

Problem b. In PyTorch, the functions model.train() and model.eval() are used to switch the model between training and evaluation modes. This distinction is important because some layers behave differently during training and inference. For example, Dropout randomly deactivates a subset of neurons during training to improve generalization, while it is disabled during evaluation so that all neurons contribute to the output. Similarly, Batch Normalization uses the current mini-batch's mean and variance for normalization during training, but relies on the running mean and variance accumulated throughout training when in evaluation mode. In HW1, however, our self-implemented library does not include layers such as Dropout or Batch Normalization, meaning that the forward computation remains identical across training and testing. As a result, explicit mode switching using model.train() or model.eval() is unnecessary in this assignment. In contrast, PyTorch is a general-purpose framework that must provide these functions to correctly handle layers whose behavior depends on the model's mode.

Problem c. The table summarizes the parameter distribution of the network, showing the number of learnable weights in each major block. As depth increases, the number of parameters grows significantly, with most concentrated in the later convolutional layers and the final fully connected layer. Batch Normalization layers contain relatively few parameters but help stabilize training. Overall, the network has about 11.69 million parameters in total.

Layer	**Conv1**	**BN1 (w,b)**	**Conv2**	**BN2 (w,b)**	**Downsample**	**Total**
conv1	9,408	64, 64	–	–	–	9,536
layer1.0	36,864	64, 64	36,864	64, 64	–	73,920
layer1.1	36,864	64, 64	36,864	64, 64	–	73,920
layer2.0	73,728	128, 128	147,456	128, 128	8,192 + (128,128)	229,888
layer2.1	147,456	128,128	147,456	128,128	–	295,296
layer3.0	294,912	256,256	589,824	256,256	32,768 + (256,256)	1,118,528
layer3.1	589,824	256,256	589,824	256,256	–	1,180,416
layer4.0	1,179,648	512,512	2,359,296	512,512	131,072 + (512,512)	5,197,584
layer4.1	2,359,296	512,512	2,359,296	512,512	–	5,744,128
fc	512,000	1,000	–	–	–	513,000
Total parameters						11,689,512

TABLE 1. Condensed layer-wise parameter summary of the network.

Problem d. Weight decay serves as a regularization technique that penalizes large parameter values, making the model smoother, less complex, and less prone to overfitting. However, the bias parameters behave differently and should not be regularized in the same way. The bias term mainly helps shift the input distribution of activation functions, ensuring that neurons operate within appropriate activation regions. It does not control model complexity and therefore does not contribute to overfitting.

If L2 regularization is applied to the bias term, the bias will be forced toward zero. This effectively shifts all activation functions back toward the origin and reduces the model's ability to adapt to inputs with non-zero means. As a result, the model may lose flexibility and perform worse on tasks requiring non-centered input distributions.

Weight decay essentially adds a penalty term to the loss function:

$$L' = L + \frac{\alpha}{2}\|w\|_2^2$$

If the bias term is also regularized, the modified loss becomes:

$$L' = L + \frac{\alpha}{2}(\|w\|_2^2 + b^2)$$

When the input data are not zero-centered, the model must learn a non-zero bias to correct the mean offset. Weight decay, by pushing the bias toward zero, prevents proper adjustment of this shift, leading to suboptimal training and larger validation error.

Problem e. Overall, the BN affine parameters provide per-channel normalization and adaptation, while the majority of learnable parameters are concentrated in convolutional and linear weight

matrices that determine the model's representational capacity.

Category	**Representative Parameters (Shape)**
BatchNorm affine parameters	bn1 (w,b) (64,)
	layer1.{0,1}.bn{1,2} (w,b) (64,)
	layer2.{0,1}.bn{1,2} (w,b) (128,)
	layer2.0.downsample.1 (w,b) (128,)
	layer3.{0,1}.bn{1,2} (w,b) (256,)
	layer3.0.downsample.1 (w,b) (256,)
	layer4.{0,1}.bn{1,2} (w,b) (512,)
	layer4.0.downsample.1 (w,b) (512,)
Bias parameters (Conv/FC)	fc.bias (1000,)
Other parameters (Conv/FC weights)	conv1.weight (64, 3, 7, 7)
	layer1.{0,1}.conv{1,2}.weight (64, 64, 3, 3)
	layer2.{0,1}.conv{1,2}.weight (128, *, 3, 3)
	layer2.0.downsample.0.weight (128, 64, 1, 1)
	layer3.{0,1}.conv{1,2}.weight (256, *, 3, 3)
	layer3.0.downsample.0.weight (256, 128, 1, 1)
	layer4.{0,1}.conv{1,2}.weight (512, *, 3, 3)
	layer4.0.downsample.0.weight (512, 256, 1, 1)
	fc.weight (1000, 512)
Summary	Total BN params: 40, Bias params: 1, Other params: 21

TABLE 2. Condensed parameter summary of the network. Grouped by BatchNorm, bias, and convolution/FC weights.

Problem f. Figure illustrates the effects of various image augmentation techniques applied to the original image `philly_original.jpg`. The augmentations include geometric transformations such as shearing along the X and Y axes (`ShearX(sx=15°)`, `ShearY(sy=15°)`), translation in both directions (`TranslateX(tx=170px0.10W)`, `TranslateY(ty=128px0.10H)`), and rotation by `30°`. Additionally, several color and intensity adjustments were performed, including `Brightness(factor=1.5)`,`Color/Saturation(factor=1.5)`,`Contrast(factor=1.5)`, and `Sharpness(factor=2.0)`. Further pixel-level augmentations such as `Posterize(bits=4)`, `Solarize(threshold=128)`, and `Equalize()` were applied to enhance visual diversity. These augmentations simulate different lighting, viewpoint, and color conditions, thereby improving the robustness and generalization ability of the trained model.

FIGURE 1. Visualization of image augmentation operations and their effects.

Solution 2 (Time spent: 1 hours)**.** Your solution goes here.

(a) Non-convexity. Consider the objective

$$f(A, B) = \|X - AB\|_F^2.$$

Let us take the scalar case $m = n = r = 1$, so that $x \in \mathbb{R}$ and $a, b \in \mathbb{R}$. Then $f(a, b) = (x - ab)^2$. Choose $x = 0$, and two points $(a_1, b_1) = (0, 2)$ and $(a_2, b_2) = (2, 0)$. We have

$$f(a_1, b_1) = f(a_2, b_2) = 0,$$

but their midpoint $(1, 1)$ satisfies $f(1, 1) = 1 > 0 = \frac{1}{2}f(a_1, b_1) + \frac{1}{2}f(a_2, b_2)$. Hence f is not convex in the joint variable (A, B).

(b) Global optimum and SVD.. Let the singular value decomposition (SVD) of X be

$$X = U\Sigma V^\top, \qquad \Sigma = \mathrm{diag}(\sigma_1, \sigma_2, \ldots, \sigma_p),$$

where $p = \min(m, n)$ and $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_p \geq 0$. Let U_r, V_r contain the first r singular vectors and $\Sigma_r = \mathrm{diag}(\sigma_1, \ldots, \sigma_r)$. By the Eckart–Young–Mirsky theorem, the best rank-r approximation of X is

$$X_r = U_r \Sigma_r V_r^\top,$$

and it satisfies

$$\min_{\mathrm{rank}(AB) \leq r} \|X - AB\|_F^2 = \|X - X_r\|_F^2 = \sum_{i=r+1}^{p} \sigma_i^2.$$

Therefore any factorization A, B such that $AB = X_r$ achieves the global minimum, for example

$$A^* = U_r \Sigma_r^{1/2}, \qquad B^* = \Sigma_r^{1/2} V_r^\top,$$

or equivalently $A^* = U_r \Sigma_r,\ B^* = V_r^\top$.

(c) Uniqueness. The solution is generally not unique.

- For any invertible $R \in \mathbb{R}^{r \times r}$,

  $$A' = A^* R, \qquad B' = R^{-1} B^*,$$

  gives the same product $A'B' = AB = X_r$.
- If the singular values satisfy $\sigma_r = \sigma_{r+1}$, the optimal subspace U_r, V_r itself is not unique: any orthogonal rotation within the degenerate subspace yields another valid optimum.

When $\sigma_r > \sigma_{r+1}$, the optimal matrix X_r is unique, but its factors A, B remain non-unique up to the above linear transformation.

Solution 3 (Time spent: 3 hours)**.** Your solution goes here.

Problem a. We trained the CIFAR-10 CNN for 100 epochs with SGD and Nesterov momentum 0.9, weight decay 10^{-3}, data augmentation (horizontal flip, crop, padding, color/contrast jitter), dropout, and batch normalization. The learning rate schedule followed 0.1 for epochs 1–40, 0.01 for epochs 41–80, and 0.001 for epochs 81–100. The loss and error curves show two clear breakpoints aligned with the scheduled decays: a rapid decrease in the first 40 epochs, a second improvement after epoch 40, and a final refinement after epoch 80. The gap between training and validation curves remains moderate, and the best validation error falls to about 8–9%, satisfying the target below 10%. Occasional validation spikes early on are consistent with heavy augmentation and small-batch stochasticity; both curves stabilize after the first decay.

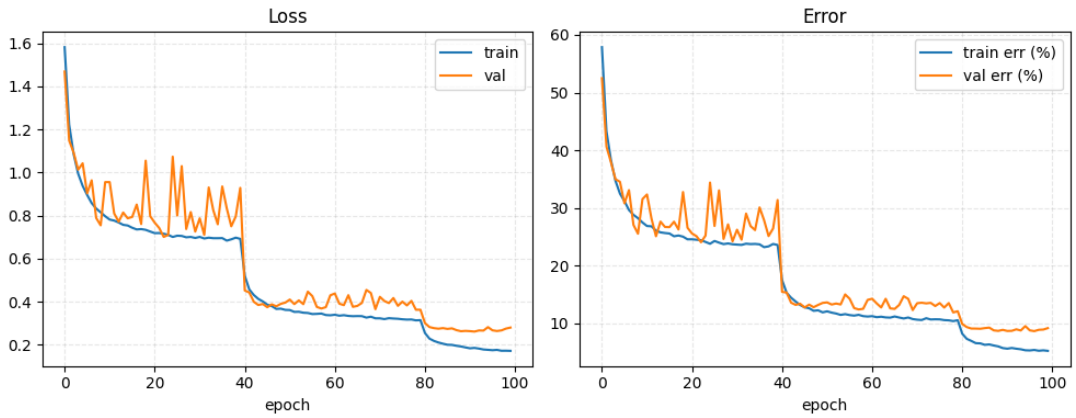
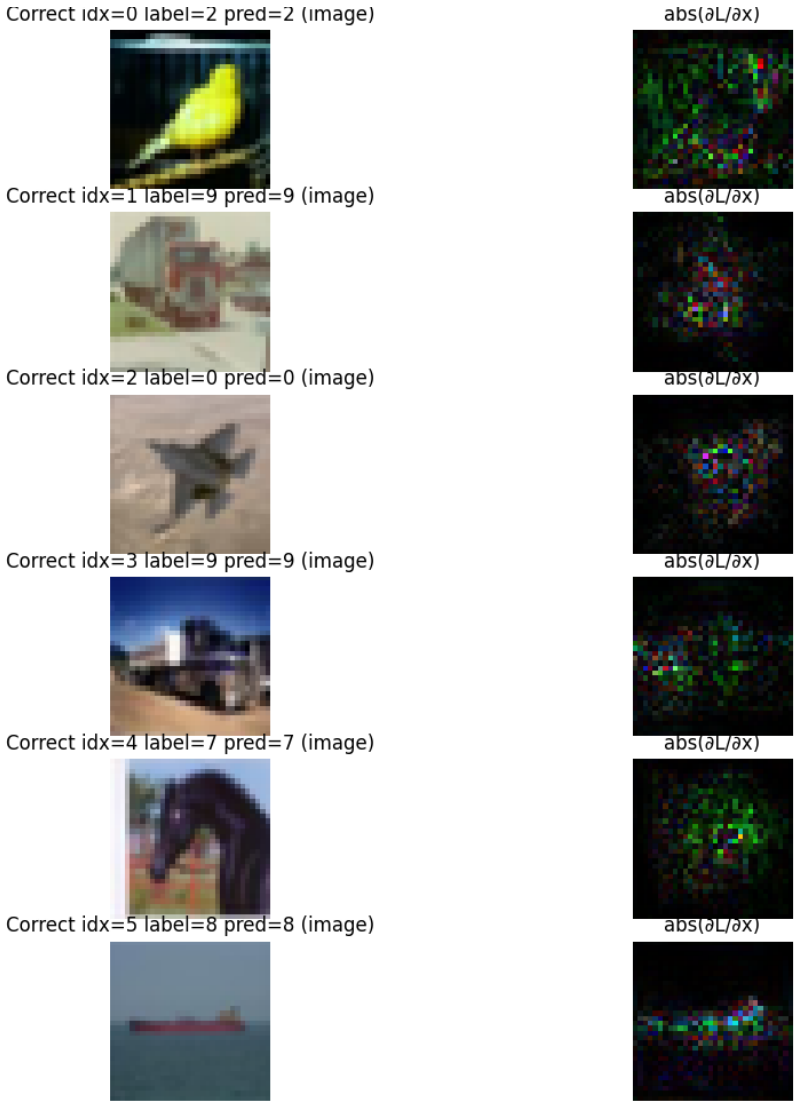
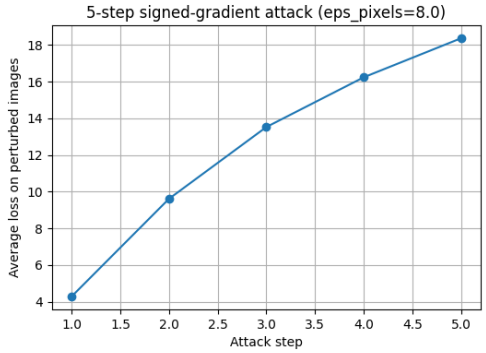


FIGURE 2. Training and Validation Loss.

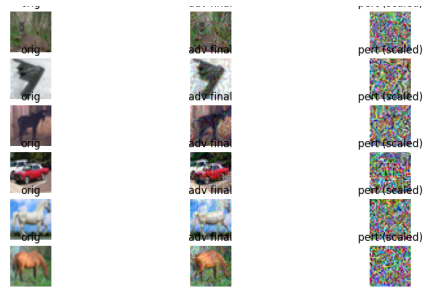
Problem b. Figure 3a shows that, for correctly classified examples, the absolute input gradients tend to concentrate around object contours and semantically relevant regions; these gradients indicate which pixels the model relies on for its decision. For misclassified examples the gradients are more diffuse, suggesting the model's decision boundary is less focused and more sensitive to distributed noise. Figure 3b displays the effect of a 5-step signed-gradient attack with $\epsilon = 8$ (pixel units): the average loss on the attacked mini-batch increases monotonically with attack steps, showing that iterative signed updates effectively increase the model loss. Correspondingly, model accuracy drops across attack steps (from A% to B% on this mini-batch). Figure 3c provides visual examples: the adversarial perturbations are small in amplitude yet sufficient to change the predicted label, which illustrates the model's vulnerability to carefully crafted, low-magnitude perturbations.



(A) Input gradient visualization. Left column: original images (denormalized). Right column: per-image normalized absolute gradients $|\partial L/\partial x|$ scaled to $[0, 1]$. Top rows show correctly classified examples; bottom rows show misclassified examples. Gradients for correctly classified images concentrate on object contours, whereas gradients for misclassified images are more diffuse.



(B) Five-step signed-gradient attack ($\epsilon = 8$ pixel units). Average loss on a mini-batch (y-axis) versus attack step (x-axis). The loss increases monotonically with step, indicating the attack successfully increases model confusion.



(C) Example adversarial images. Columns show original image, final adversarial image, and the scaled perturbation for visibility. Perturbation amplitudes are visually small but flip model predictions.

FIGURE 3. Analysis of input gradients and adversarial attacks.

Problem c. We evaluated the network on the validation set under a single-step Fast Gradient Sign Method (FGSM) attack with $\epsilon = 8/255$. The clean validation accuracy was 91.37%, while the accuracy on the adversarially perturbed images dropped sharply to 20.72%, a decrease of approximately 70.65 percentage points. This large performance gap demonstrates that the network is highly vulnerable to even small, imperceptible adversarial perturbations, highlighting the sensitivity of standard CNNs to gradient-based attacks.

Problem d. Although the human brain is also a high-dimensional classifier with a vast number of neurons, it is generally more robust to small, imperceptible perturbations than artificial neural networks. Humans are not easily fooled by pixel-level changes that are nearly invisible, since perception integrates contextual, semantic, and hierarchical information beyond raw pixel values. However, humans can be deceived by larger or structured perturbations that exploit perceptual biases rather than low-level noise. Examples include optical illusions such as the checker shadow illusion, where identical colors appear different due to contextual lighting cues, or ambiguous images like the Rubin vase and the "dress" image that appears blue–black or white–gold depending on global illumination assumptions. In these cases, the perturbations are not small in the pixel-wise ℓ_∞ sense, but instead act on higher-level perceptual priors. Thus, while both humans and neural networks can be fooled, the mechanisms and scales of their adversarial vulnerabilities are fundamentally different.

Solution 4 (Time spent: 3 hours)**.** Your solution goes here.

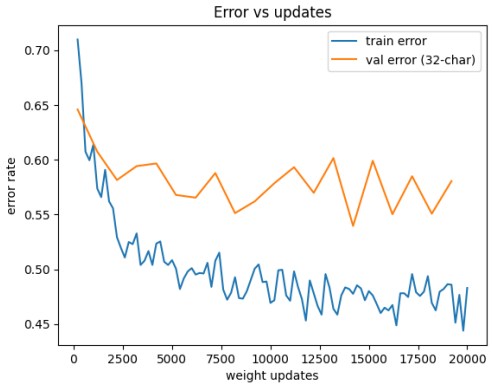
Problem a. Each character in the text, including letters, digits, punctuation, spaces, and newlines, was treated as a unique token. The vocabulary size m was obtained by computing the number of unique characters in the dataset:

$$m = \texttt{len(set(all_chars))} = 107.$$

Each input sequence consisted of 32 consecutive characters from the text. Every character was represented as a one-hot vector of dimension 107, resulting in an input tensor of shape `torch.Size([32, 107])`. This encoding directly maps each character to a unique binary basis vector, enabling the model to learn character-level dependencies without any preprocessing or tokenization. Neural networks are typically robust to noisy or unclean data, so no further text cleaning was performed.

Problem b. We trained a single-layer tanh RNN that predicts the next character over a vocabulary of size m using a softmax output and cross-entropy loss. Each training step consumes a 32-character sequence encoded as one-hot vectors and optimizes the average negative log-likelihood of the next character.

The learning curves in Figure 4 show that the training loss drops steadily from about 2.6 to roughly 1.55 and the mini-batch error decreases from about 71% to about 45% over 20k weight updates. The validation curves, evaluated every 1k updates on a future 32-character sequence, initially improve (loss from about 2.6 to about 2.25 and error from about 64% to about 56%), then fluctuate around a plateau with small oscillations. This behavior is typical for character-level models trained with small batches: the training objective continues to decrease while the validation error stabilizes, indicating that the model has learned useful short-range dependencies but begins to overfit mildly after the early phase. Overall, validation tracks training closely with a persistent but moderate gap, consistent with proper optimization and limited overfitting.



(A) error rate versus weight updates (train and validation)

(B) cross-entropy loss versus weight updates (train and validation)

FIGURE 4. RNN learning curves on character prediction. Validation metrics are computed every 1000 updates on a held-out 32-character sequence and averaged over the mini-batch.

Problem d. We initialized the hidden state to zero and rolled the RNN forward by feeding the previous prediction $\hat{y}_t$ back as the next input x_{t+1}. The vocabulary size was 117 and the model was loaded from the saved checkpoint. Below we show two decoding strategies. Greedy decoding collapses to a short loop with limited diversity, while sampling at temperature $T = 1.0$ produces more varied text that preserves punctuation and line structure, but remains semantically nonsensical.

Greedy decoding:

```
The that shall be the world the world the world the world the world
the world the world the world the world the world the world the world
the world the world the world the world the world the world the world
the world the world the world the world the world the world the wo
```

Sampling (T = 1.0):

```
The fealy doth the bottle paid come of stain
Your Mingardun.

PROSPERO.
Ay, but his valiage.

I'll tell.

WARWICK.
O, I shoulded false us to patiend. But tellost, I had this matter's a
downAre my plain, nor sented, am, Cowsents yfillow and there
```

```
 Scene II. Command I thank thee.

[_Exit, thereffirity,
```

Observation: the model captures basic syntax and punctuation (capitalization, line breaks, speaker tags), yet greedy decoding quickly repeats phrases, and even with sampling the content lacks coherent semantics— typical behavior for a small character-level RNN trained briefly.

APPENDIX

APPENDIX A. RESNET 18 PARAM. COUNTING

```
import torch
import torchvision.models as models

model = models.resnet18()

total_params = 0
print(f"{'Layer':40s} {'Parameters':>15s}")
print("-" * 60)
for name, param in model.named_parameters():
    if param.requires_grad:
        num_params = param.numel()
        total_params += num_params
        print(f"{name:40s} {num_params:15,d}")

print("-" * 60)
print(f"{'Total':40s} {total_params:15,d}")
```

APPENDIX B. RESNET 18 PARAM. SEPERATELY COUNTING

```
import torch
import torchvision.models as models
import torch.nn as nn

model = models.resnet18()


bn_params, bias_params, other_params = [], [], []

for name, param in model.named_parameters():
    if isinstance(dict(model.named_modules()).get(name.split('.')[0],
        None), nn.BatchNorm2d) or 'bn' in name or 'downsample.1' in name
        :
        bn_params.append((name, param.shape))
    elif 'bias' in name and param.requires_grad:
        bias_params.append((name, param.shape))
    else:
        other_params.append((name, param.shape))

print("\n(i) BatchNorm affine parameters:")
```

```
for n, s in bn_params:
    print(f"  {n:<40} {tuple(s)}")

print("\n(ii) Bias parameters of Conv/FC layers:")
for n, s in bias_params:
    print(f"  {n:<40} {tuple(s)}")

print("\n(iii) Other parameters (Conv/FC weights, etc.):")
for n, s in other_params:
    print(f"  {n:<40} {tuple(s)}")

print("\nSummary:")
print(f"  Total BN params: {len(bn_params)}")
print(f"  Total bias params: {len(bias_params)}")
print(f"  Total other params: {len(other_params)}")
```

Appendix C. Data Augmentation

```
import argparse, os
from PIL import Image
import numpy as np
from torchvision.transforms import functional as F
import torchvision.utils as vutils
import torch


def load_image(path=None):

        img = Image.open(path).convert("RGB")
        name = os.path.splitext(os.path.basename(path))[0]

        return img, name

def pil_to_tensor(img):
    return F.to_tensor(img)

def tensor_to_pil(t):
    return F.to_pil_image(t.clamp(0,1))

def ensure_dir(p):
    os.makedirs(p, exist_ok=True)
```

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--img", type=str, default="philly.png", help="
        path to input image (RGB)")
    parser.add_argument("--out", type=str, default="aug_out", help="
        output directory")
    args = parser.parse_args()

    ensure_dir(args.out)
    img, base = load_image(args.img)

    W, H = img.size

    img_t = pil_to_tensor(img)

    results = []

    # 1) ShearX
    sx = 15.0
    im = F.affine(img, angle=0.0, translate=(0,0), scale=1.0, shear=(sx,
         0), interpolation=Image.BILINEAR, fill=0)
    results.append((f"ShearX(sx={sx}  )", im))

    # 2) ShearY
    sy = 15.0
    im = F.affine(img, angle=0.0, translate=(0,0), scale=1.0, shear=(0,
        sy), interpolation=Image.BILINEAR, fill=0)
    results.append((f"ShearY(sy={sy}  )", im))

    # 3) TranslateX
    tx = int(0.10 * W)
    im = F.affine(img, angle=0.0, translate=(tx, 0), scale=1.0, shear
        =(0.0, 0.0), interpolation=Image.BILINEAR, fill=0)
    results.append((f"TranslateX(tx={tx} p x 0 .10W)", im))

    # 4) TranslateY
    ty = int(0.10 * H)
    im = F.affine(img, angle=0.0, translate=(0, ty), scale=1.0, shear
        =(0.0, 0.0), interpolation=Image.BILINEAR, fill=0)
    results.append((f"TranslateY(ty={ty} p x 0 .10H)", im))
```

```
# 5) Rotate
angle = 30.0
im = F.rotate(img, angle=angle, interpolation=Image.BILINEAR, fill
    =0)
results.append((f"Rotate(angle={angle}  )", im))

# 6) Brightness
b_factor = 1.5
im = F.adjust_brightness(img, b_factor)
results.append((f"Brightness(factor={b_factor})", im))

# 7) Color (Saturation)
c_factor = 1.5
im = F.adjust_saturation(img, c_factor)
results.append((f"Color/Saturation(factor={c_factor})", im))

# 8) Contrast
ctr_factor = 1.5
im = F.adjust_contrast(img, ctr_factor)
results.append((f"Contrast(factor={ctr_factor})", im))

# 9) Sharpness
sh_factor = 2.0
im = F.adjust_sharpness(img, sh_factor)
results.append((f"Sharpness(factor={sh_factor})", im))

# 10) Posterize
bits = 4
im = F.posterize(img, bits)
results.append((f"Posterize(bits={bits})", im))

# 11) Solarize
thr = 128
im = F.solarize(img, threshold=thr)
results.append((f"Solarize(threshold={thr})", im))

# 12) Equalize
im = F.equalize(img)
results.append(("Equalize()", im))


grid_tensors = []
```

```
    orig_name = f"{base}_original.jpg"
    img.save(os.path.join(args.out, orig_name))
    grid_tensors.append(pil_to_tensor(img))
    print(f"Saved: {orig_name}")

    for title, im in results:
        fname = f"{base}_{title.replace('/','-').replace('  ','deg').
            replace('   ','~').replace(' ','_').replace('(','[').replace
            (')',']')}.jpg"
        im.save(os.path.join(args.out, fname))
        print(f"Saved: {fname}")
        grid_tensors.append(pil_to_tensor(im))

    grid = vutils.make_grid(torch.stack(grid_tensors, dim=0), nrow=4,
        padding=4)
    grid_img = tensor_to_pil(grid)
    grid_path = os.path.join(args.out, f"{base}_augment_grid_4x3.jpg")
    grid_img.save(grid_path)
    print(f"Saved grid: {grid_path}")

    txt_path = os.path.join(args.out, f"{base}_augment_params.txt")
    with open(txt_path, "w", encoding="utf-8") as f:
        f.write("Augment Parameters\n")
        f.write(f"Original: {orig_name}\n")
        for title, _ in results:
            f.write(f"{title}\n")
    print(f"Wrote params: {txt_path}")
```

APPENDIX D. NN TRAIN

```
import torch, torch.nn as nn, torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import torchvision
import matplotlib.pyplot as plt
from collections import defaultdict
import os, math, random
import numpy as np

def seed_everything(seed=2025):
```

```
    random.seed(seed); np.random.seed(seed); torch.manual_seed(seed);
        torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True; torch.backends.cudnn.
        benchmark = False
seed_everything(42)

# model
class View(nn.Module):
    def __init__(self,o): super().__init__(); self.o = o
    def forward(self,x): return x.view(-1, self.o)

def convbn(ci,co,ksz,s=1,pz=0):
    return nn.Sequential(
        nn.Conv2d(ci,co,ksz,stride=s,padding=pz), #          : Conv ->
            ReLU -> BN
        nn.ReLU(True),
        nn.BatchNorm2d(co)
    )

class allcnn_t(nn.Module):
    def __init__(self, c1=96, c2=192):
        super().__init__()
        d = 0.5
        self.m = nn.Sequential(
            nn.Dropout(0.2),
            convbn(3,c1,3,1,1),
            convbn(c1,c1,3,1,1),
            convbn(c1,c1,3,2,1),      # 32->16
            nn.Dropout(d),
            convbn(c1,c2,3,1,1),
            convbn(c2,c2,3,1,1),
            convbn(c2,c2,3,2,1),      # 16->8
            nn.Dropout(d),
            convbn(c2,c2,3,1,1),
            convbn(c2,c2,3,1,1),
            convbn(c2,10,1,1),
            nn.AvgPool2d(8),          # -> [N,10,1,1]
            View(10)                  # -> [N,10]
        )
    def forward(self,x): return self.m(x)

# CIFAR-10 mean/std
```

```
mean = (0.4914, 0.4822, 0.4465)
std  = (0.2470, 0.2435, 0.2616)

train_tf = transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean, std),
])
test_tf = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean, std),
])

root = "./data"
train_set = datasets.CIFAR10(root, train=True,  download=True, transform
    =train_tf)
test_set  = datasets.CIFAR10(root, train=False, download=True, transform
    =test_tf)

batch_size = 128
num_workers = 2
train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True
    ,  num_workers=num_workers, pin_memory=True)
val_loader   = DataLoader(test_set,  batch_size=batch_size, shuffle=
    False, num_workers=num_workers, pin_memory=True)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = allcnn_t().to(device)

def accuracy(logits, targets):
    return (logits.argmax(1) == targets).float().mean().item()

bn_params, bias_params, other_params = [], [], []
for name, p in model.named_parameters():
    if not p.requires_grad: continue
    if ('bn' in name) or ('running_mean' in name) or ('running_var' in
        name):
         if name.endswith('weight') or name.endswith('bias'):
             bn_params.append(p)
    elif name.endswith('bias'):
```

```
        bias_params.append(p)
    else:
        other_params.append(p)

# weight decay = 1e-3
base_wd = 1e-3
optimizer = torch.optim.SGD([
    {'params': other_params, 'weight_decay': base_wd},
    {'params': bn_params,   'weight_decay': 0.0},
    {'params': bias_params, 'weight_decay': 0.0},
], lr=0.1, momentum=0.9, nesterov=True)

#  l r 0 -40:0.1, 40-80:0.01, 80-100:0.001
def lr_schedule(epoch):
    if epoch < 40: return 0.1
    elif epoch < 80: return 0.01
    else: return 0.001

criterion = nn.CrossEntropyLoss()

# ---------- Train / Validate ----------
history = defaultdict(list)
best_val_acc, best_state = 0.0, None

epochs = 100
for epoch in range(epochs):
    # set LR
    for g in optimizer.param_groups:
        g['lr'] = lr_schedule(epoch)

    # Train
    model.train()
    tr_loss_sum, tr_acc_sum, n_tr = 0.0, 0.0, 0
    for x,y in train_loader:
        x, y = x.to(device, non_blocking=True), y.to(device,
            non_blocking=True)
        optimizer.zero_grad(set_to_none=True)
        logits = model(x)
        loss = criterion(logits, y)
        loss.backward()
        optimizer.step()
```

```
        bs = x.size(0)
        tr_loss_sum += loss.item()*bs
        tr_acc_sum  += accuracy(logits, y)*bs
        n_tr += bs

    tr_loss = tr_loss_sum / n_tr
    tr_acc  = tr_acc_sum  / n_tr
    tr_err  = 1.0 - tr_acc

    # Val
    model.eval()
    va_loss_sum, va_acc_sum, n_va = 0.0, 0.0, 0
    with torch.no_grad():
        for x,y in val_loader:
            x, y = x.to(device, non_blocking=True), y.to(device,
                non_blocking=True)
            logits = model(x)
            loss = criterion(logits, y)
            bs = x.size(0)
            va_loss_sum += loss.item()*bs
            va_acc_sum  += accuracy(logits, y)*bs
            n_va += bs
    va_loss = va_loss_sum / n_va
    va_acc  = va_acc_sum  / n_va
    va_err  = 1.0 - va_acc

    history['train_loss'].append(tr_loss)
    history['train_err'].append(tr_err)
    history['val_loss'].append(va_loss)
    history['val_err'].append(va_err)

    if va_acc > best_val_acc:
        best_val_acc = va_acc
        best_state = {k:v.cpu().clone() for k,v in model.state_dict().
            items()}

    print(f"Epoch {epoch+1:03d}/{epochs} | "
          f"LR {optimizer.param_groups[0]['lr']:.3g} | "
          f"Train loss {tr_loss:.4f} err {tr_err*100:.2f}% | "
          f"Val loss {va_loss:.4f} err {va_err*100:.2f}%")

os.makedirs("checkpoints", exist_ok=True)
```

```
torch.save(best_state, "checkpoints/allcnn_best.pth")
print(f"Best val acc: {best_val_acc*100:.2f}%, model saved to
    checkpoints/allcnn_best.pth")


fig, ax = plt.subplots(1,2, figsize=(10,4))
ax[0].plot(history['train_loss'], label='train')
ax[0].plot(history['val_loss'],   label='val')
ax[0].set_title("Loss"); ax[0].set_xlabel("epoch"); ax[0].legend(); ax
    [0].grid(True, ls='--', alpha=0.3)

ax[1].plot([e*100 for e in history['train_err']], label='train err (%)')
ax[1].plot([e*100 for e in history['val_err']],   label='val err (%)')
ax[1].set_title("Error"); ax[1].set_xlabel("epoch"); ax[1].legend(); ax
    [1].grid(True, ls='--', alpha=0.3)
plt.tight_layout(); plt.show()
```

Appendix E. NN gradient visualization and adversarial attack

```
import torch, torch.nn as nn
import torchvision
from torchvision import transforms, datasets
import matplotlib.pyplot as plt
import numpy as np
import os
from collections import defaultdict

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

# Model definition
class View(nn.Module):
    def __init__(self,o): super().__init__(); self.o=o
    def forward(self,x): return x.view(-1,self.o)

def convbn(ci,co,ksz,s=1,pz=0):
    return nn.Sequential(
        nn.Conv2d(ci,co,ksz,stride=s,padding=pz),
        nn.ReLU(True),
        nn.BatchNorm2d(co)
    )
```

```
class allcnn_t(nn.Module):
    def __init__(self, c1=96, c2=192):
        super().__init__()
        d = 0.5
        self.m = nn.Sequential(
            nn.Dropout(0.2),
            convbn(3,c1,3,1,1),
            convbn(c1,c1,3,1,1),
            convbn(c1,c1,3,2,1),
            nn.Dropout(d),
            convbn(c1,c2,3,1,1),
            convbn(c2,c2,3,1,1),
            convbn(c2,c2,3,2,1),
            nn.Dropout(d),
            convbn(c2,c2,3,1,1),
            convbn(c2,c2,3,1,1),
            convbn(c2,10,1,1),
            nn.AvgPool2d(8),
            View(10)
        )
    def forward(self,x): return self.m(x)

# Dataset
mean = (0.4914, 0.4822, 0.4465)
std  = (0.2470, 0.2435, 0.2616)

test_tf = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean, std),
])

test_set = datasets.CIFAR10("./data", train=False, transform=test_tf,
   download=True)
test_loader = torch.utils.data.DataLoader(test_set, batch_size=100,
   shuffle=True, num_workers=0,pin_memory=True)

mean_t = torch.tensor(mean).view(3,1,1).to(device)
std_t  = torch.tensor(std).view(3,1,1).to(device)

def denormalize(tensor):
    return (tensor * std_t) + mean_t
```

```
def normalize_from_uint8(img_uint8):
    img = img_uint8.float() / 255.0
    return (img - mean_t) / std_t


model = allcnn_t().to(device)
ckpt_path = "checkpoints/allcnn_best.pth"
if os.path.exists(ckpt_path):
    print("Loading checkpoint:", ckpt_path)
    state = torch.load(ckpt_path, map_location='cpu')
    # state is likely a state_dict
    try:
        model.load_state_dict(state)
    except Exception:
        # handle if saved dict was {k: v.cpu()} earlier
        model.load_state_dict({k: v.to(device) for k, v in state.items()
            })

model.eval()

# -------------------------
# Part A: compute input gradients dx for some correctly and incorrectly
    classified images
# -------------------------
# get one minibatch (100 samples)
xs, ys = next(iter(test_loader))
xs, ys = xs.to(device), ys.to(device)

# forward to get preds
with torch.no_grad():
    logits = model(xs)
preds = logits.argmax(dim=1)
correct_mask = (preds == ys)
incorrect_mask = ~correct_mask
print(f"Batch size {xs.size(0)}: {correct_mask.sum().item()} correct, {
    incorrect_mask.sum().item()} incorrect")

def visualize_grads_for_indices(indices, title_prefix):
    imgs = []
    grads = []
    titles = []
```

```
    for idx in indices:
        x = xs[idx:idx+1].detach().clone().requires_grad_(True)  # keep
            grad for input
        y = ys[idx:idx+1]
        logits = model(x)
        loss = nn.CrossEntropyLoss()(logits, y)
        # zero grads for input
        if x.grad is not None:
            x.grad.zero_()
        # compute gradients wrt input
        loss.backward()
        dx = x.grad.detach().cpu().squeeze(0)  # shape [C,H,W]
        # Convert grad to visualization: absolute, per-channel ->
            combine to rgb by max or mean
        grad_img = dx.abs()
        # normalize gradient to [0,1] for visualization
        grad_vis = grad_img.numpy()
        grad_vis = grad_vis / (grad_vis.max() + 1e-12)
        # denormalize the original image for display (to [0,1])
        img_display = denormalize(x.detach().cpu().squeeze(0)).cpu().
            numpy()
        img_display = np.clip(img_display, 0.0, 1.0)
        imgs.append(np.transpose(img_display, (1,2,0)))
        grads.append(np.transpose(grad_vis, (1,2,0)))
        titles.append(f"{title_prefix} idx={int(idx)} label={int(y.item
            ())} pred={int(logits.argmax(1).item())}")
    # plot
    n = len(imgs)
    if n==0:
        return
    fig, axs = plt.subplots(n, 2, figsize=(6,3*n))
    if n==1:
        axs = np.expand_dims(axs, 0)
    for i in range(n):
        axs[i,0].imshow(imgs[i])
        axs[i,0].axis('off')
        axs[i,0].set_title(titles[i] + " (image)")
        axs[i,1].imshow(grads[i])
        axs[i,1].axis('off')
        axs[i,1].set_title("abs( L / x )")
    plt.tight_layout()
    plt.show()
```

```
correct_indices = [i for i in range(xs.size(0)) if correct_mask[i].item
    ()][:6]
incorrect_indices = [i for i in range(xs.size(0)) if incorrect_mask[i].
    item()][:6]

print("Visualizing gradients for correctly classified examples:",
    correct_indices)
visualize_grads_for_indices(correct_indices, "Correct")

print("Visualizing gradients for misclassified examples:",
    incorrect_indices)
visualize_grads_for_indices(incorrect_indices, "Incorrect")

# -------------------------
# Part B: 5-step signed gradient attack on the batch
# -------------------------
eps_pixels = 8.0
eps_unit = eps_pixels / 255.0
eps_per_channel = (eps_unit / std_t).to(device)

print("eps per channel in normalized space (approx):", eps_per_channel.
    squeeze().cpu().numpy())

num_steps = 5
batch_size = xs.size(0)
loss_fn = nn.CrossEntropyLoss(reduction='none')

x_adv = xs.detach().clone()   # already normalized
x_adv.requires_grad = True


def clamp_in_normalized_space(x_norm):
    x_unnorm = denormalize(x_norm)
    x_unnorm = torch.clamp(x_unnorm, 0.0, 1.0)
    return (x_unnorm - mean_t) / std_t

# arrays to record average loss per step
avg_loss_per_step = []
# optionally also record fraction of images whose prediction changed
acc_per_step = []
```

```
for step in range(num_steps):
    # ensure gradients from previous step are cleared
    if x_adv.grad is not None:
        x_adv.grad.zero_()
    # forward
    logits_adv = model(x_adv)
    losses = loss_fn(logits_adv, ys)   # per-sample losses
    loss_mean = losses.mean()
    # backward to compute  L / x  for the batch; here we use mean loss
        to aggregate
    loss_mean.backward()
    dx = x_adv.grad.detach()   # shape [B,C,H,W]
    # sign of gradient
    dx_sign = dx.sign()
    x_adv = (x_adv + eps_per_channel * dx_sign).detach()
    x_adv = clamp_in_normalized_space(x_adv)
    x_adv.requires_grad = True

    # record average loss after this perturbation (use model in eval)
    with torch.no_grad():
        logits_now = model(x_adv)
        losses_now = loss_fn(logits_now, ys)
        avg_loss = losses_now.mean().item()
        avg_loss_per_step.append(avg_loss)
        acc_now = (logits_now.argmax(dim=1) == ys).float().mean().item()
        acc_per_step.append(acc_now)
    print(f"Step {step+1}/{num_steps}: avg loss = {avg_loss:.4f},
       accuracy = {acc_now*100:.2f}%")

# Plot average loss vs step
plt.figure(figsize=(6,4))
plt.plot(range(1,num_steps+1), avg_loss_per_step, marker='o')
plt.xlabel("Attack step")
plt.ylabel("Average loss on perturbed images")
plt.title(f"5-step signed-gradient attack (eps_pixels={eps_pixels})")
plt.grid(True)
plt.show()

# Plot accuracy vs step
plt.figure(figsize=(6,4))
```

```
plt.plot(range(1,num_steps+1), [1 - a for a in acc_per_step], marker='o'
    )  # plot error %
plt.xlabel("Attack step")
plt.ylabel("Error rate on perturbed images")
plt.title(f"Error rate vs attack step (eps_pixels={eps_pixels})")
plt.grid(True)
plt.show()

# -------------------------
# Visualize a few adversarial examples and their perturbations (
    difference)
# -------------------------
# pick first 6 images
kvis = min(6, batch_size)
x_orig = xs[:kvis].cpu()
x_final = x_adv.detach().cpu()[:kvis]
# compute perturbation in pixel-space for visualization
pert_pixel = (denormalize(x_final) - denormalize(x_orig)).numpy()  # in
    [ -1, 1 ] approx
# normalize to displayable magnitude
def to_display(img_tensor):
    img = img_tensor.numpy()
    img = np.transpose(img, (1,2,0))  # H W C
    img = np.clip(img, 0.0, 1.0)
    return img

fig, axs = plt.subplots(kvis, 3, figsize=(9,3*kvis))
for i in range(kvis):
    axs[i,0].imshow(to_display(denormalize(x_orig[i]).cpu()))
    axs[i,0].axis('off'); axs[i,0].set_title("orig")
    axs[i,1].imshow(to_display(denormalize(x_final[i]).cpu()))
    axs[i,1].axis('off'); axs[i,1].set_title("adv final")
    # perturbation: scale for visibility
    p = pert_pixel[i]
    p_vis = (p - p.min()) / (p.max()-p.min()+1e-12)
    axs[i,2].imshow(np.transpose(p_vis, (1,2,0)))
    axs[i,2].axis('off'); axs[i,2].set_title("pert (scaled)")
plt.tight_layout(); plt.show()
```

Appendix F. NN gradient visualization and adversarial attack

APPENDIX G. NN 1 STEP GRADIENT ADVERSARIAL ATTACK

```
import torch, torch.nn as nn
import torchvision
from torchvision import transforms, datasets
import matplotlib.pyplot as plt
import numpy as np
import os
from collections import defaultdict
from tqdm import tqdm


device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

# Model definition
class View(nn.Module):
    def __init__(self,o): super().__init__(); self.o=o
    def forward(self,x): return x.view(-1,self.o)

def convbn(ci,co,ksz,s=1,pz=0):
    return nn.Sequential(
        nn.Conv2d(ci,co,ksz,stride=s,padding=pz),
        nn.ReLU(True),
        nn.BatchNorm2d(co)
    )

class allcnn_t(nn.Module):
    def __init__(self, c1=96, c2=192):
        super().__init__()
        d = 0.5
        self.m = nn.Sequential(
            nn.Dropout(0.2),
            convbn(3,c1,3,1,1),
            convbn(c1,c1,3,1,1),
            convbn(c1,c1,3,2,1),
            nn.Dropout(d),
            convbn(c1,c2,3,1,1),
            convbn(c2,c2,3,1,1),
            convbn(c2,c2,3,2,1),
            nn.Dropout(d),
            convbn(c2,c2,3,1,1),
```

```
            convbn(c2,c2,3,1,1),
            convbn(c2,10,1,1),
            nn.AvgPool2d(8),
            View(10)
        )
    def forward(self,x): return self.m(x)


# Dataset
mean = (0.4914, 0.4822, 0.4465)
std  = (0.2470, 0.2435, 0.2616)

test_tf = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean, std),
])

test_set = datasets.CIFAR10("./data", train=False, transform=test_tf,
   download=True)
test_loader = torch.utils.data.DataLoader(test_set, batch_size=100,
   shuffle=True, num_workers=0,pin_memory=True)

mean_t = torch.tensor(mean).view(3,1,1).to(device)
std_t  = torch.tensor(std).view(3,1,1).to(device)

def denormalize(tensor):
    return (tensor * std_t) + mean_t

def normalize_from_uint8(img_uint8):
    img = img_uint8.float() / 255.0
    return (img - mean_t) / std_t

# Load model

model = allcnn_t().to(device)
ckpt_path = "checkpoints/allcnn_best.pth"
if os.path.exists(ckpt_path):
    print("Loading checkpoint:", ckpt_path)
    state = torch.load(ckpt_path, map_location='cpu')
    # state is likely a state_dict
    try:
        model.load_state_dict(state)
```

```
    except Exception:
        # handle if saved dict was {k: v.cpu()} earlier
        model.load_state_dict({k: v.to(device) for k, v in state.items()
            })

model.eval()


# -------------------------
# Part C: 1-step signed gradient attack on the batch
# -------------------------

@torch.no_grad()
def clamp_in_normalized_space(x_norm):
    x_unnorm = denormalize(x_norm)         # -> [0,1] (approx), shape [B
        ,C,H,W]
    x_unnorm = torch.clamp(x_unnorm, 0.0, 1.0)
    return (x_unnorm - mean_t) / std_t

def accuracy_on_clean(model, loader, device):
    model.eval()
    correct, total = 0, 0
    for x, y in tqdm(loader, desc="Evaluating clean accuracy"):
        x, y = x.to(device), y.to(device)
        logits = model(x)
        pred = logits.argmax(1)
        correct += (pred == y).sum().item()
        total += y.numel()
    return correct / total


def accuracy_on_fgsm1(model, loader, device, eps_pixels=8.0):
    model.eval()
    eps_unit = eps_pixels / 255.0
    eps_per_channel = (eps_unit / std_t).to(device)
    correct_adv, total = 0, 0
    loss_fn = nn.CrossEntropyLoss()

    for x, y in tqdm(loader, desc="Running 1-step FGSM attack"):
        x, y = x.to(device), y.to(device)

        x_adv = x.detach().clone().requires_grad_(True)
```

```
        logits = model(x_adv)
        loss = loss_fn(logits, y)
        if x_adv.grad is not None:
            x_adv.grad.zero_()
        loss.backward()

        g = x_adv.grad.detach()
        x_adv = (x_adv + eps_per_channel * g.sign()).detach()
        x_adv = clamp_in_normalized_space(x_adv)

        with torch.no_grad():
            logits_adv = model(x_adv)
            pred_adv = logits_adv.argmax(1)
            correct_adv += (pred_adv == y).sum().item()
            total += y.numel()

    return correct_adv / total


# ----- run & report -----
clean_acc = accuracy_on_clean(model, test_loader, device)
fgsm1_acc = accuracy_on_fgsm1(model, test_loader, device, eps_pixels
   =8.0)

print(f"Clean accuracy: {clean_acc*100:.2f}%")
print(f"1-step FGSM (  =8/255) accuracy: {fgsm1_acc*100:.2f}%")
print(f"Drop: {(clean_acc - fgsm1_acc)*100:.2f} pp")
```

APPENDIX H. RNN VOCABULARY

```
import torch

def text_to_onehot(seq, char_to_idx):
    vocab_size = len(char_to_idx)
    onehots = torch.zeros(len(seq), vocab_size)
    for t, ch in enumerate(seq):
        if ch in char_to_idx:
            onehots[t, char_to_idx[ch]] = 1.0
        else:
            pass
    return onehots
```

```
with open("Shakespeare.txt", "r", encoding="utf-8") as f:
    text = f.read()

vocab = sorted(list(set(text)))
m = len(vocab)
print(f"Vocabulary size: {m}")

char_to_idx = {ch: i for i, ch in enumerate(vocab)}
idx_to_char = {i: ch for i, ch in enumerate(vocab)}



# test
sample_seq = text[:32]
X = text_to_onehot(sample_seq, char_to_idx)

print("Input sequence:")
print(repr(sample_seq))
print("\nOne-hot tensor shape:", X.shape)
print( X.sum(dim=1)[:5])
```

Appendix I. RNN Train

```
import math, random, os, io
import torch, torch.nn as nn
from torch.utils.data import Dataset, DataLoader
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("device:", device)

# Config
train_files = [
    "war_and_peace.txt",
    "Siddhartha.txt"
]
test_files  = [
    "Shakespeare.txt"
]

SEQ_LEN   = 32
BATCH     = 64
```

```
HIDDEN    = 256
LR        = 2e-3
UPDATES   = 20000
PRINT_EVERY = 200
EVAL_EVERY  = 1000


# build train/test data set and vocab.
def read_all(paths):
    buf = []
    for p in paths:
        with io.open(p, "r", encoding="utf-8", errors="ignore") as f:
            buf.append(f.read())
    return "".join(buf)

train_text = read_all(train_files)
test_text  = read_all(test_files)

vocab = sorted(list(set(train_text)))
UNK = "<UNK>"
if any(ch not in vocab for ch in test_text):
    vocab = [UNK] + vocab

stoi = {ch:i for i,ch in enumerate(vocab)}
itos = {i:ch for ch,i in stoi.items()}
V = len(vocab)
print(f"Vocabulary size: {V}")

def encode(s):
    if UNK in stoi:
        return [stoi.get(ch, stoi[UNK]) for ch in s]
    else:
        return [stoi[ch] for ch in s if ch in stoi]

# one-hot encoding
def to_one_hot(x_idx, V):
    B, T = x_idx.shape
    oh = torch.zeros(B, T, V, device=x_idx.device)
    oh.scatter_(2, x_idx.unsqueeze(-1), 1.0)
    return oh
```

```
# Sliding window to dataloader
class CharSeqDataset(Dataset):
    def __init__(self, text, seq_len):
        self.idx = encode(text)
        self.seq_len = seq_len
        self.n = len(self.idx) - (seq_len + 1)
        self.n = max(self.n, 0)
    def __len__(self):
        return self.n
    def __getitem__(self, i):
        x = torch.tensor(self.idx[i:i+self.seq_len], dtype=torch.long)
        y = torch.tensor(self.idx[i+1:i+1+self.seq_len], dtype=torch.
            long)
        return x, y

train_ds = CharSeqDataset(train_text, SEQ_LEN)
val_ds   = CharSeqDataset(test_text,  SEQ_LEN)
train_loader = DataLoader(train_ds, batch_size=BATCH, shuffle=True,
    drop_last=True)
val_loader   = DataLoader(val_ds,   batch_size=BATCH, shuffle=False,
    drop_last=True)



# Rnn model
class CharRNN(nn.Module):
    def __init__(self, vocab_size, hidden):
        super().__init__()
        self.rnn = nn.RNN(input_size=vocab_size,
                          hidden_size=hidden,
                          num_layers=1,
                          nonlinearity='tanh',
                          batch_first=True)
        self.fc  = nn.Linear(hidden, vocab_size)
    def forward(self, x_oh, h0=None):
        out, hT = self.rnn(x_oh, h0)
        logits = self.fc(out)
        return logits, hT

model = CharRNN(V, HIDDEN).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LR)
```

```python
# evaluation
def batch_ce_and_err(logits, y):
    B,T,V = logits.shape
    loss = criterion(logits.reshape(B*T, V), y.reshape(B*T))
    with torch.no_grad():
        pred = logits.argmax(dim=-1)
        acc  = (pred == y).float().mean().item()
        err  = 1.0 - acc
    return loss, err

def evaluate_on_val32():
    model.eval()
    with torch.no_grad():
        x,y = next(iter(val_loader))
        x = x.to(device); y = y.to(device)
        x_oh = to_one_hot(x, V)
        logits, _ = model(x_oh)
        loss, err = batch_ce_and_err(logits, y)
    model.train()
    return loss.item(), err

# training loop
ckpt_dir = "checkpoints"
os.makedirs(ckpt_dir, exist_ok=True)

best_val_loss = float("inf")
last_ckpt     = os.path.join(ckpt_dir, "last.pt")
best_ckpt     = os.path.join(ckpt_dir, "best.pt")
tr_losses, tr_errs, val_losses, val_errs, steps = [], [], [], [], []
step = 0
model.train()
while step < UPDATES:
    for x, y in train_loader:
        x = x.to(device); y = y.to(device)
        x_oh = to_one_hot(x, V)
        logits, _ = model(x_oh)
        loss, err = batch_ce_and_err(logits, y)
```

```
        optimizer.zero_grad()
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)  #

        optimizer.step()

        step += 1
        if step % PRINT_EVERY == 0:
            tr_losses.append(loss.item()); tr_errs.append(err); steps.
                append(step)
            print(f"[{step}] train loss {loss.item():.3f} | err {err:.3f
                }")

        if step % EVAL_EVERY == 0:
            vloss, verr = evaluate_on_val32()
            if vloss < best_val_loss:
                best_val_loss = vloss
                torch.save({
                    "step": step,
                    "model_state_dict": model.state_dict(),
                    "optimizer_state_dict": optimizer.state_dict(),
                    "val_loss": vloss,
                    "vocab": vocab,
                    "stoi": stoi, "itos": itos
                }, best_ckpt)
                print(f"       Saved BEST to {best_ckpt} | step={step} |
                    val_loss={vloss:.4f}")

            val_losses.append(vloss); val_errs.append(verr)
            print(f"              val32 loss {vloss:.3f} | err {verr:.3f
                }")

        if step >= UPDATES:
            break

# plot
plt.figure()
plt.plot(steps, tr_losses, label="train loss")
plt.plot(steps[::EVAL_EVERY//PRINT_EVERY], val_losses, label="val loss
    (32-char)")
plt.xlabel("weight updates"); plt.ylabel("loss"); plt.legend(); plt.
    title("Loss vs updates")
```

```
plt.show()
plt.savefig('loss.png')

plt.figure()
plt.plot(steps, tr_errs, label="train error")
plt.plot(steps[::EVAL_EVERY//PRINT_EVERY], val_errs, label="val error
    (32-char)")
plt.xlabel("weight updates"); plt.ylabel("error rate"); plt.legend();
    plt.title("Error vs updates")
plt.show()
plt.savefig('error.png')
```

Appendix J. RNN Inference

```
import math, random, os, io
import torch, torch.nn as nn
from torch.utils.data import Dataset, DataLoader
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("device:", device)

# Config
train_files = [
    "war_and_peace.txt",
    "Siddhartha.txt"
]
test_files  = [
    "Shakespeare.txt"
]

SEQ_LEN   = 32
BATCH     = 64
HIDDEN    = 256
LR        = 2e-3
UPDATES   = 20000
PRINT_EVERY = 200
EVAL_EVERY  = 1000


# build train/test data set and vocab.
def read_all(paths):
```

```
    buf = []
    for p in paths:
        with io.open(p, "r", encoding="utf-8", errors="ignore") as f:
            buf.append(f.read())
    return "".join(buf)

train_text = read_all(train_files)
test_text  = read_all(test_files)

vocab = sorted(list(set(train_text)))
UNK = "<UNK>"
if any(ch not in vocab for ch in test_text):
    vocab = [UNK] + vocab

stoi = {ch:i for i,ch in enumerate(vocab)}
itos = {i:ch for ch,i in stoi.items()}
V = len(vocab)
print(f"Vocabulary size: {V}")

def encode(s):
    if UNK in stoi:
        return [stoi.get(ch, stoi[UNK]) for ch in s]
    else:
        return [stoi[ch] for ch in s if ch in stoi]

# one-hot encoding
def to_one_hot(x_idx, V):
    B, T = x_idx.shape
    oh = torch.zeros(B, T, V, device=x_idx.device)
    oh.scatter_(2, x_idx.unsqueeze(-1), 1.0)
    return oh


# Sliding window to dataloader
class CharSeqDataset(Dataset):
    def __init__(self, text, seq_len):
        self.idx = encode(text)
        self.seq_len = seq_len
        self.n = len(self.idx) - (seq_len + 1)
        self.n = max(self.n, 0)
    def __len__(self):
        return self.n
```

```
    def __getitem__(self, i):
        x = torch.tensor(self.idx[i:i+self.seq_len], dtype=torch.long)
        y = torch.tensor(self.idx[i+1:i+1+self.seq_len], dtype=torch.
            long)
        return x, y

train_ds = CharSeqDataset(train_text, SEQ_LEN)
val_ds   = CharSeqDataset(test_text,  SEQ_LEN)
train_loader = DataLoader(train_ds, batch_size=BATCH, shuffle=True,
   drop_last=True)
val_loader   = DataLoader(val_ds,   batch_size=BATCH, shuffle=False,
   drop_last=True)



# Rnn model
class CharRNN(nn.Module):
    def __init__(self, vocab_size, hidden):
        super().__init__()
        self.rnn = nn.RNN(input_size=vocab_size,
                          hidden_size=hidden,
                          num_layers=1,
                          nonlinearity='tanh',
                          batch_first=True)
        self.fc  = nn.Linear(hidden, vocab_size)
    def forward(self, x_oh, h0=None):
        out, hT = self.rnn(x_oh, h0)
        logits = self.fc(out)
        return logits, hT

model = CharRNN(V, HIDDEN).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LR)




ckpt_path = "checkpoints/best.pt"
ckpt = torch.load(ckpt_path, map_location=device)
print(f" Loaded checkpoint from {ckpt_path}")

vocab = ckpt["vocab"]
```

```
stoi = ckpt["stoi"]
itos = ckpt["itos"]
UNK = "<UNK>" if "<UNK>" in stoi else None
V = len(vocab)

model = CharRNN(V, HIDDEN).to(device)
model.load_state_dict(ckpt["model_state_dict"])
model.eval()



#eval
def sample_text(seed=" ", length=400, temperature=1.0, greedy=False):

    h = torch.zeros(1, 1, HIDDEN, device=device)

    for ch in seed[:-1]:
        x_idx = torch.tensor([[stoi.get(ch, stoi.get(UNK, 0))]], device=
            device)
        x_oh = to_one_hot(x_idx, V)
        _, h = model(x_oh, h)

    cur = seed[-1] if seed else " "
    out = [c for c in seed]


    for _ in range(length):
        x_idx = torch.tensor([[stoi.get(cur, stoi.get(UNK, 0))]], device
            =device)
        x_oh = to_one_hot(x_idx, V)
        logits, h = model(x_oh, h)
        logits = logits[0, 0] / max(temperature, 1e-6)

        if greedy:
            nxt_idx = int(torch.argmax(logits).item())
        else:
            probs = torch.softmax(logits, dim=-1)
            nxt_idx = int(torch.multinomial(probs, 1).item())

        cur = itos[nxt_idx]
        out.append(cur)
```

```
    return "".join(out)

print("\n=== Generated text (greedy) ===")
print(sample_text(seed="The ", length=300, greedy=True))

print("\n=== Generated text (temp=1.0, sampling) ===")
print(sample_text(seed="The ", length=300, temperature=1.0, greedy=False
    ))
```